

Programmering i skolen

Hvorfor, hva og hvordan?
Algoritmisk tenkning og problemløsning

Algoritmisk tenkning



Forord

Dette veiledningsheftet er et supplement for programmeringskurset for ENT3R-mentorer arrangert for ENT3R.

Med stor respekt vil jeg gjerne berømme arbeidet som legges ned av Nasjonalt senter for Realfagsrekruttering. Den gode jobben ENT3R-mentorer gjør for å inspirere ungdommene til å drive med realfag, og ikke minst for å skape et konkurransedyktig, bærekraftig og likestilt samfunn, er beundringsverdig.

Anbefaling til leseren

Når du skal lære et nytt emne er det ikke nok å pugge eller lære seg å få tak i hvordan ting skal gjøres. Det er like viktig å spørre seg hvorfor metoden er som den er, og forstå hvordan ting henger sammen. Da blir det lettere å lære. Jo bedre du forstår hvert delemne, desto lettere er det å bruke den til å løse aktuelle problemer. Du får da god erfaring til å anvende den i andre fagfelt.

Takk

Jeg har hatt samarbeid med ENT3R snart i femten år, og vil gjerne herved takke ENT3R-mentorene om med sitt viktige arbeid kan hjelpe elevene å komme et steg nærmere til sine drømmer. Jeg vil spesielt sende en stor takk til Marthina Sætre Vahedian, Lorentz T. Syslak, Thea Tyssøy Jensen, og alle ENT3R-mentorene i Bergen for korrekturlesing av teksten og innspill. En stor takk rettes til Jonas Øivind Wiborg og Alexander Westre Skog ved Bergen kommune og mine kollegaer ved HVL, for å lese gjennom dette heftet, og ha hjulpet å forbedre både teksten og innholdet.

Jeg vil gjerne takke Utdanningsdirektoratet som har godkjent å bruke deres notater og figurer (<https://www.udir.no/>).

Bergen, 18. februar 2024

Amir M. Hashemi

Eventuelle kommentarer eller meldinger om feil i notatene tas imot med glede: ahas@hvl.no.

ISBN 9788294170012
© Forfatterne 2025–2030

Innhold

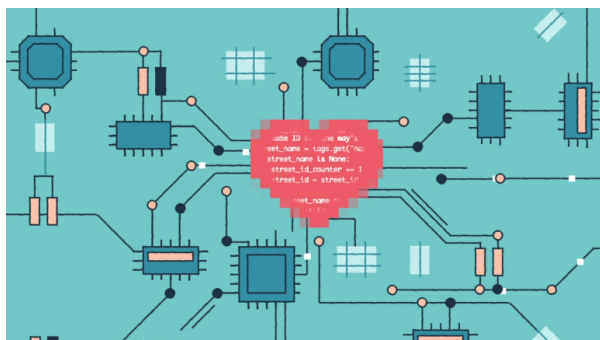
1	Hvorfor programmering i skolen?	1
1.1	Kompetanse for fremtiden	1
1.2	Programmering i skolen	1
1.3	Hvorfor er programmering så viktig?	3
1.4	Nettressurser	4
2	Hvordan programmering i skolen?	5
2.1	Viktige begrep i programmering	6
2.2	Koding og programmering	7
2.3	Programmering og programmeringsdidaktikk	8
3	Algoritmisk tenkning	9
3.1	Hva handler algoritmisk tenkning om?	9
3.2	Fremme og implementere algoritmisk tenkning	12
3.3	Eksempler på algoritmisk tenkning i forskjellige fag	13
4	Programmering og problemløsning	16
4.1	Hvorfor programmering?	16
4.2	Hva er et program?	16
4.3	Eksempler programmering for matematikk programmering for modellering	18
5	Programmering som støtte for matematikkundervisningen	28
	Fasit	33

Kapittel 1

Hvorfor programmering i skolen?

1.1 Kompetanse for fremtiden

De siste årene har teknologi hatt en betydelig innvirkning i måten vi lever på og hvordan vi forholder oss til hverandre i samfunnet. Mobile enheter med et lynraskt internett resulterer i tilgjengelighet og kommunikasjon som aldri før har vært mulig. Underholdning er i dag tilgjengelig til enhver tid og til ethvert sted, samtidig som at det er personlig tilpasset etter egne ønsker og behov. Teknologi har også påvirket måten vi lærer på, søker etter informasjon og tilegner oss kunnskap. Dette skaper nye utfordringer for pedagogenes arbeidsmetoder, der målet er å støtte elevenes utvikling av fagkunnskaper og grunnleggende ferdigheter. Kompetanse er sammensatt av ferdigheter, evner, kunnskap, forståelse, egenskaper, holdninger og verdier. De ulike faktorene forsterker og komplementerer hverandre. Det innebærer at kompetanse er både sammensatt av de enkelte komponentene og resultatet av samspillet mellom disse. Kompetansebehovet knyttet til næringsliv og samfunnsbehov kan klassifiseres i



- behov for breddekompetanse
- behov for spiss-/dybdekompetanse på ulike felt

I den første kategorien kan nevnes behovet for å øke digitalkompetansen i hele befolkningen og behovet for den generelle digitale forståelsen i samfunnsviktige yrker. I den andre kategorien, eksempelvis kunstig intelligens eller dataanalyse, kreves det at høyere utdanningsinstitusjoner, universiteter og høyskoler, tilrettelegger sine studier for å kunne imøtekomme behovet i samarbeid med næringslivet.

1.2 Programmering i skolen

Ifølge regjeringens nye læreplaner skal alle barn lære å programmere. Behovet for flere realister har vært stort lenge. Lærdommen etter en rekke realfagsstrategier fra ulike land over hele verden er at ungdom velger det de selv foretrekker og opplever til å være interessant.

Det å velge realfag gir flere valgmuligheter. Programmering i skolen begrunnes ofte med at det er en nødvendig kompetanse for å lære, arbeide og leve i dagens og morgendagens samfunn. Til dette refereres det ofte til hva som er kompetanser og ferdigheter for det 21. århundre. Slike kompetanser er definert på en rekke ulike måter, så vel som i forskjellige rapporter og dokumenter. Når det er sagt, ser vi samtidig kjennetegn og beskrivelser som går igjen uavhengig av hvilken definisjon eller rapport en velger å forholde seg til. De fleste forsøk på å beskrive disse ferdighetene inneholder begreper som kreativitet, innovasjon, kritisk tenkning, metakognisjon (lære å lære), kommunikasjon, samarbeid, digital kompetanse, digital dannelse, medborgerskap, og karriere og arbeidsliv [11].

Den europeiske unionen (EU) har satt programmering på sin *Digital Agenda for Europe* og oppfordrer utdanningsministre i medlemslandene til å fremme programmering i skolen. De begrunner satsingen med at programmering er en viktig ferdighet som fremmer kreativitet, samarbeid over geografiske avstander og å kommunisere via et felles språk. Dypere forståelse av underliggende prosesser og systemer, evne til å beherske logisk tenkning, samt ferdigheter i å være skapende og produserende har alltid vært viktige i læring og utdanning. Utviklingen av det teknologirike samfunnet vi har i dag og i fremtiden, utfordrer både måten vi lærer på og hvilke kompetanser som blir viktige. Å skape meningsfulle læringsprosesser, utvikle skapende evner og dyp forståelse krever andre tilnærminger i teknologirike omgivelser. Å skape og produsere digitalt krever blant annet forståelse og kompetanse i programmering.

I de siste årene har koding og programmering fått mye oppmerksomhet i læringsmiljøer. Å fremme algoritmisk tenkning for å tilegne seg problemløsningsevne har blitt en verdensomspennende trend. I flere land som eksempelvis Spania og Frankrike er læreplaner fornyet for å satse på koding og programmering, i tillegg til bruk av droner og roboter på barneskoletrinnet.

Det spanske departementet for utdanning og yrkesopplæring har lansert *School of computational thinking* for studieåret 2018-2019. Hovedmålet er å trene lærere i beregningstenkning og teknologi. På denne måten vil de innlemme disse ferdighetene i undervisningsressursene i klasserommet, ettersom beregningstenkning defineres som en grunnleggende ferdighet i det 20. århundre og kan hjelpe studenter til å nå nye jobbmuligheter, samt forbedre deres samfunnsdeltakelse som borgere.



Satsing på fremtiden er intet nytt tema, og det har lenge vært predikert at teknologi vil prege morgendagens samfunnsliv. Avslutningsvis til delkapittelet inkluderer vi utsagn fra store, viktige og innflytelsesrike stemmer:

In the coming years, we should build on that progress, by ... offering every student the hands-on computer science and math classes that make them job-ready on Day 1.

- President Obama, 2016

Learning to write programs stretches your mind, and helps you think better, creates a way of thinking about things that I think is helpful in all domains.

- Bill Gates

De nye læreplanene skal gi elevene et bedre grunnlag for å reflektere, være kritiske, skapende, utforskende og kreative.

- Sanner (kunnskaps- og integreringsminister 2018-2020, pressemelding 18.03.2019)

1.3 Hvorfor er programmering så viktig?

Nedenfor er det listet opp tre punkter som kan gi en kortfattet beskrivelse på viktigheten av programmeringen i skolen:

1. Å fremme realfag, spesielt matematikk som nyttig verktøy (rekruttering til STEM)
2. Å skape interesse for faget hos ungdommene
3. Å holde ungdommene interessert og engasjert i løpet av studietiden (senke frafall)

I de senere årene har flere offentlige utredninger pekt på behovet for hvordan skolen og skolefagene må endres for å møte fremtidens kompetansebehov. Argumentet for programmering i skolen knyttes gjerne til nødvendige ferdigheter som elevene kommer til å trenge. Ved å implementere programmering inn i flere fag, vil dette kunne gi mer aktiv læring, og store muligheter for å være kreativ. Det er ingen hemmelighet at satsning på realfag har pågått i mange år uten at responsen blant elevene har vært tilsvarende stor. Programmering vil kunne gi elevene en mer spennende, aktiv og utviklende skolehverdag, samtidig som de trenes i logikk, kreativitet og bedre digitale ferdigheter.

Ifølge notat fra Udir [11] er det i hovedsak tre tilnærminger til å innføre programmering i skolen:

- Programmering som (del av) eget IKT-fag
- Programmering integrert i eksisterende fag
- Programmering som fagovergripende kompetanse i flere fag.

Videre i dette heftet vil vi gå nærmere inn på hvordan man som lærer kan støtte elevene til å utvikle kompetanser ved å integrere programmering i undervisningen, derav også på tvers av ulike fag. Før vi avslutter dette kapittelet, gis det en rask gjennomgang av nettressurser som kan være behjelpelig i arbeid for å integrere programmering på barne- og ungdomstrinnet.

1.4 Nettressurser

Det er flere nettsider som vil fremme interessen for tiltaket:

<https://code.org/>

<https://www.barefootcomputing.org/>

Micro:bit programmering med Scratch og Python:

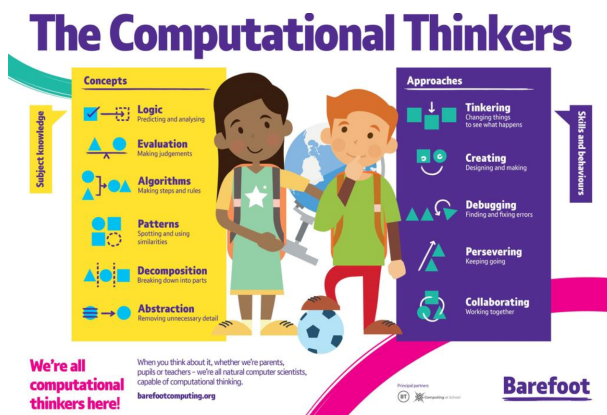
<https://makecode.microbit.org/>

Det finnes også flere gratis nettressurser som kan brukes til å tilrettelegge programmering for barnetrinnet og ungdomstrinnet på forskjellige nivåer. Eksempler på dette er Scratch og Code Studio. Begge har til fordel at de er lette å komme i gang med, de har flere gode instruksjonsaktiviteter for nybegynnere, men de har også utfordringer på høyere nivåer.

<https://scratch.mit.edu/>

<https://studio.code.org/courses>

<https://projecteuler.net/>



Kapittel 2

Hvordan programmering i skolen?

I dagens høyteknologiske og rask skiftende verden blir det stadig tydeligere at elevene må kunne tenke kritisk og løse komplekse og ikke veldefinerte problemer som relateres til hverdagen. Mens få vil argumentere for nytten av å undervise i kritisk tenkning og problemløsningsferdigheter i skolene, er det mindre enighet om hvordan du skal gjøre det, når du skal begynne, eller hvilke begreper du skal bruke når du tilrettelegger i disse viktige kompetansene.

I forrige kapittel ble det løftet frem en rekke argumenter på hvorfor vi bør implementere programmering i den norske skolen. De viktigste prinsippene bygger på det verdenskjente rammeverket *21st century skills*, som forsøker å definere hvilke egenskaper og ferdigheter som vil være nødvendig for elever å beherske i fremtiden. Ved en raskt skiftende verden som stadig preges av en høyt akselererende teknologisk utvikling, vil det være mye som tilsier et økende behov for teknologisk kompetanse i det 21.århundre. Det predikteres også behov for kompetanser innenfor problemløsning, utforskning og kritisk tenkning for å kunne imøtekomme morgendagens muligheter og utfordringer. Til tross for positive tilbakemeldinger, støtte og stor enighet om hva som vil være fremtidens kompetansebehov, er det samtidig ikke gitt en entydig beskrivelse på hvordan elevene skal lære dette i skolen. Et lignende problem kan knyttes til resultater fra høringen av fagfornyelsen til læreplanverket for matematikk [2]. Funn tilsier at norske lærere har positiv holdning til å implementere programmering i skolen, men at de samtidig opplever usikkerhet knyttet til hvordan programmering skal tilrettelegges og inkluderes i undervisningen. I dette kapitlet vil vi derfor redegjøre for hvordan programmering kan integreres i undervisningen, og introdusere begreper som er relevant i forbindelse med dette.

2.1 Viktige begrep i programmering

Algoritmisk tenkning blir i økende grad anerkjent som en viktig pedagogisk ferdighet som er karakteristisk for læring i det 21. århundre. De senere årene har ordet begrepet «koding» blitt stadig mer utbredt i dagligtale, spesielt i samtaler som omhandler hvordan barn og unge som skal lære seg å programmere. Programmering kan knyttes til alt som dreier seg om å løse et problem ved hjelp av en algoritme eller et program. Dette innebærer blant annet forarbeid, planlegge, skrive, tegne, bearbeide, undersøke og teste problemløsningsmetoden, eksperimentere, revurdere, argumentere for at algoritmen alltid virker, generalisere algoritmen til å løse en hel klasse av problemer og undersøke effektivitet. Figuren forsøker å illustrere hvordan programmering avhenger algoritmisk tenkning og at programmering er mye mer enn bare koding.

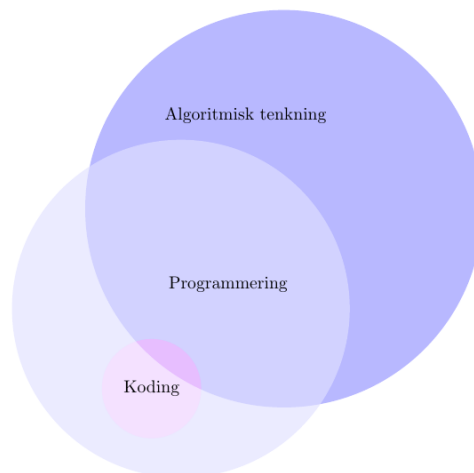


Fig. 2.1 Programmering og koding

Programmering kan knyttes til alt som dreier seg om å løse et problem ved hjelp av en algoritme eller et program. Programmering i skolen nevnes gjerne ofte i samme åndedrag som algoritmisk tenkning. Dette kommer trolig av hvordan programmering ofte beskrives om det å kommunisere ved hjelp av logikk og algoritmisk tenkning. Basert på denne definisjonen, fremkommer *koding* som selve prosessen og arbeidet med å reformulere dette inn i et bestemt *programmeringsspråk*.

Algoritmisk tenkning utgjør et viktig nøkkelp prinsipp i naturvitenskapelig praksis og matematikk. I skolen har vi derfor lenge jobbet arbeidet med å tilrettelegge for at elever skal kunne utvikle elevenes ferdigheter innenfor til algoritmisk tenkning. Begrepene algoritmisk tenkning, programmering og koding blir i enkelte tilfeller og sammenhenger brukt mye om i noen sammenhenger om hverandre. Programmering er de konkrete skrittene som er nødvendig for å kunne skrive en kode. Programmering innebærer dermed å legge opp en plan (utforme pseudokode), bryte ned problemet i delproblemer og foreta selve kodingen. For å kunne redegjøre for hvordan programmering kan integreres i undervisning, må vi først avdekke nøkkelpbegreper som er relevant i forbindelse med dette.

2.2 Koding og programmering

Koding og programmering utgjør er de to viktigste tilnærmingene innenfor programvareutviklingsvitenskapen. Vi vil derfor utdype de to begrepene nærmere i dette delkapittelet.

- **Programmering** beskrives om prosessen i å utvikle et kjørbart program på datamaskinnivå som kan implementeres uten å gi feilmeldinger. Det er prosessen med å skrive koder formelt slik at menneskelige innganger og tilsvarende maskinutganger forblir synkronisert.
- **Koding** omfatter er bare den delen av programmeringsprosessen som handler om å sende oppgaver og instruksjoner til datamaskin hva den skal gjøre. Koding i skolen dreier seg om at elevene lære elever til å løse problemer og samarbeide på kreative måter.

Visste du at vi bruker algoritmisk tenkning daglig? Dersom du tar en liten titt rundt livet ditt, så vil du fort innse at alt kan knyttes til et sett av spørsmål og svar. Hvordan kan du lage noe hvis du er sulten? Hvordan kan barna dine gå på skolen? Hvordan kan du forbedre deg selv til treningsstudioet? Hvordan kan du handle med et begrenset beløp? De fleste av våre daglige aktiviteter implementeres trinn for trinn og i henhold til hva som er førsteprioriteten. Dette gjøres ved å anvende logisk analyse, dekomponering, abstraksjon, mønstergjenkjenning og algoritmer, som alle utgjør hovedbegreper i algoritmisk tenkning.

Programmering utgjør mer enn å bare implementere allerede eksisterende algoritmer. Det handler om en prosess eller et samspill mellom problemløsning og metoder/algoritmer for å løse en hel klasse av problemer. Det å finne en generell metode utgjør den viktigste delen av prosessen, noe som gjør matematikkfaget til en naturlig arena. Dette gjelder uavhengig om digitale verktøy inngår i arbeidet eller ikke. Et viktig poeng som er verdt å se, og som gjerne ofte misforstås, handler om hva som selve målet med å lære programmering er. Fokuset bør ikke rettes mot å kunne beherske flest mulig programmeringsspråk, men heller forstå essensen bak de ulike delprosessene. Slik dannes det rom for utvikling og forbedring av ferdigheter innenfor logisk og algoritmisk tenkning, problemløsning og modellering. En slik forståelse utgjør en viktig forutsetning for å lykkes med å implementere programmering i lærings- og undervisningssammenhenger.

Kompetansemål i læreplan matematikk 1-10 trinn [14] og matematikk fellesfag Vg1 teoretisk [15] kan hjelpe oss bedre å forstå hvilke kompetansemål forventes at elevene skal utvikle og dette kan hjelpe skolen og lærere å sette konkrete og klare læringsmål og planlegge hvordan de skal tilrettelegge programmeringen realfag og programfag. Med kjerneelementer mener vi både det viktigste innholdet, og det elevene må lære for å kunne mestre og bruke faget. De fastsatte kjerneelementene beskriver sentrale begreper, metoder, tenkemåter, kunnskapsområder og uttrykksformer, kan bidra å forstå hva som danner grunnlaget for utviklingen av læreplanene [9].

Algoritmisk tenkning kan betraktes som en generell kompetanse, og gjennom algoritmisk tenkemåte skal grunnlaget for programmering dannes. Ved arbeid med algoritmisk problemløsning kan lærere tilrettelegge at elevene få muligheter til å utvikle seg innenfor alle kjerneelementene, og læreren får mulighet til å vurdere elevenes kompetanse og få innsikt i hvordan de bruker sin kunnskap til å argumentere, reflektere, analysere og resonnere.

2.3 Programmering og programmeringsdidaktikk

Målet med dette heftet er å belyse hvordan vi best kan planlegge og tilrettelegge for matematikk og fysikk i skolen, og undersøke hvordan vi kan innlemme programmering i disse fagene.

I de neste kapitlene vil begrepene algoritmisk tenkning, problemløsning, modellering, programmering og sammenhengen mellom disse bli presentert. Med dette ønsker vi å bidra til å gi et faglig grunnlag for programmering, i tillegg til å støtte læreres kompetanse innenfor programmeringsdidaktikk.

Figuren nedenfor forsøker å illustrere hvordan man kan utforme og sette opp læringsmål ved hjelp av å ta utgangspunkt i kompetansemålene for hvert fag. Deretter kan det utformes en liste av delemner hvor programmering kan integreres. Dette kan bidra til å støtte og hjelpe lærerens arbeid med planlegging og tilrettelegging av undervisning hvor programmering inngår. Samtidig vil det trolig oppleves til å være mer eller mindre overkommelig å sette opp en slik liste, basert på hvilket fag man underviser i. Matematikk og fysikk er fag med kompetansemål som egner seg godt for innlæring gjennom programmering [11]. Programmering i skolen handler mye om logisk tenkning og problemløsning, noe som lett kan overføres til arbeidsoppgaver i flere fag. Algoritmetenkning er nyttig for matematikk og fysikkfagene, men det er også viktig å lære seg å tenke stegvis i form av sekvenser og konsekvenser i andre fag.

Programmeringsspråket, som stort sett brukes i ungdomsskolen og videregående, er *Python*, mens barneskolen bruker *Scratch*. Men det er viktig å lære tankegangen bak programmering, ikke programmeringsspråkene i seg selv. I de neste kapitlene diskuterer vi hvordan programmering kan brukes for å øke forståelsen i de ulike fagene.

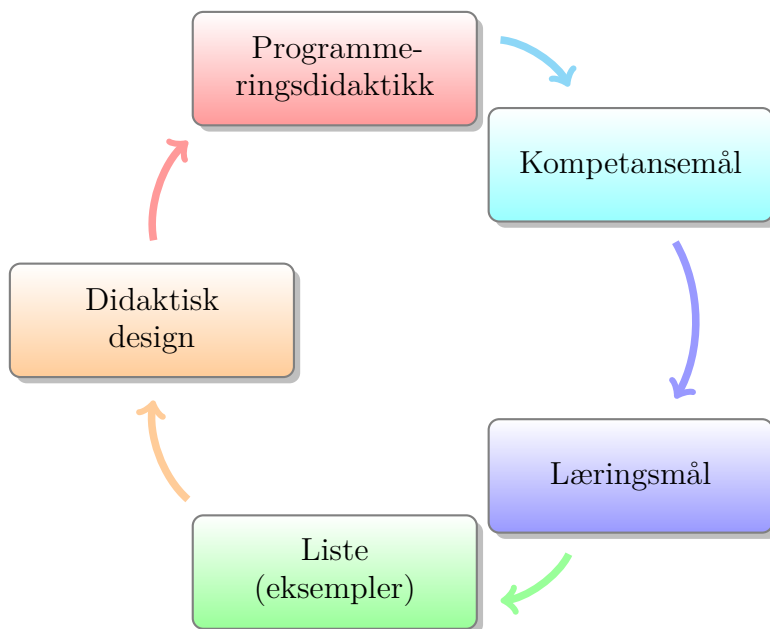


Fig. 2.2. Programmeringsdidaktikk med utgangspunkt i kompetansemål

Kapittel 3

Algoritmisk tenkning

Problemløsning i matematikk og andre fag handler om at elevene utvikler en metode for å løse et problem de ikke har kjennskap til fra tidligere. Algoritmisk tenkning utgjør en viktig del av prosessen med å utvikle strategier og framgangsmåter for å bryte opp og løse problemer på en systematisk måte. Dette innebærer også å foreta en vurdering av hvorvidt digitale verktøy kan bidra til å løse problemet på en effektiv og hensiktsmessig måte. Problemløsning handler også samtidig om å analysere og omforme kjente og ukjente problemer, løse dem og vurdere om løsningene er gyldige. Algoritmisk tenkning knyttes i større grad til de mentale prosessene som finner sted når vi forsøker, og handler om måten en må tenke på for å oversette et problem uttrykt i ord til en logisk rekke av operasjoner som kan kodes inn i en datamaskin.

I dette kapittelet går vi nærmere inn på hva algoritmisk tenkning innebærer, da forståelse rundt begrepet utgjør en viktig del av arbeid med programmering.

3.1 Hva handler algoritmisk tenkning om?

Begrepet *algoritmisk tenkning* er hentet fra det engelske uttrykket *computational thinking* og ble først anvendt av Seymour Papert i boka *Mindstorms: Children, computers, and powerful ideas* fra 1980. I artikkelen *Computational Thinking* [16] beskriver Professor Jeanette Wing, algoritmisk tenkning som en ferdighet og tilnærming som er relevant for alle, også utenom datavitenskapen. *Algoritmisk tenkning* (computational thinking) er en problemløsningsmetode [10]. Som tidligere nevnt, innebærer algoritmisk tenkning å tilnærme seg problemer på en systematisk måte, både når vi formulerer hva det er vi ønsker å løse og når vi foreslår mulige løsninger. Å tenke algoritmisk er å anvende de ulike stegene for å løse et problem, og å kunne bruke sin teknologiske kompetanse for å få en datamaskin til å løse (deler av)



Figur 3.1: Konsepter og arbeidsmåter for algoritmisk tenkning fra Barefoot (UK) tilpasset av Utdanningsdirektoratet.

problemet. I dette ligger også en forståelse av hva slags problemer/oppgaver som kan løses med teknologi og hva som bør overlates til mennesker. En av mange definisjoner går ut på at algoritmisk tenkning er *tankeprosessen* knyttet til å formulere et problem og dets løsning(er) på en slik måte at det kan løses av en informasjonsprosesser (menneske eller maskin). Algoritmisk tenkning er altså et sett med konsepter og fremgangsmåter knyttet til prosessen fra man blir klar over et problem til man har funnet en løsning på problemet. Algoritmisk tenkning handler om å bryte ned problemet i løsbare delproblemer, generalisere og modellere løsninger og finne stegene man må igjennom for å komme til en endelig løsning.

Algoritmisk og beregningstankegang

Som tidligere nevnt, er algoritmisk tankegang eller beregningstankegang (computational thinking) er et sett med ferdigheter og prosesser som gjør det mulig å navigere i komplekse problemer. Algoritmisk tankegang involverer å dele opp komplekse problemer til en serie mindre, mer løsbare problemer (dekomponering). Hvert av disse delproblemene kan så behandles separat ved å vurdere hvordan liknende problemer har vært løst tidligere (mønsterkjennelse) og eliminerer de mindre viktige detaljene, fokuserer på de viktige detaljene og ignorerer irrelevant informasjon (abstraksjon). Til sist utformes stegvise instruksjoner og regler for å løse hvert av de mindre problemene (algoritmer).

Algoritmisk tenkning er et kart fra nysgjerrighet til forståelse og en bro fra problem til problemløsning/modellering

Beregningstankeprosessen starter med data som input og et forsøk på å utlede mening og løsning fra det. Resultatet er ikke bare et svar, men en prosess, en løsningsmetode og løsningsstrategi, for å nå frem til nettopp dette resultatet. For å være et kart mot forståelse, planlegger beregningstankene, reisen for å sikre at prosessen kan replikeres . På den måten kan man sikre kvalitet i prosessen ved at andre også kan ta den i bruk, så vel som å uthente lærdom og kunnskap fra den. Det er nettopp på dette tidspunktet at beregningstenkning ofte strømmer inn i algoritmisk tenkning.

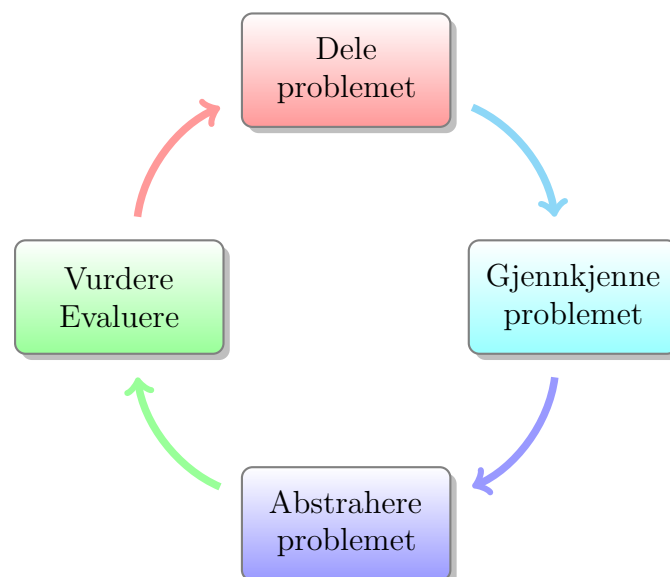


Fig. 3.2 Konsepter i algoritmisk tenkning.

Dette utvider forståelsen og definisjonen på hva begrepet algoritmisk tenkning innebærer: det handler om å utvikle en trinnvis prosess for å løse problemet slik at arbeidet kan replikeres

av mennesker eller datamaskiner. Her henvises det til den britiske organisasjonen Barefoot Computing [1] som tar utgangspunkt i seks elementer/steg/trinn som inngår i algoritmisk tenkning [11]:

- Logikk - analysere og forutse (hypoteser)
- Algoritmer - lage steg-for-steg-beskrivelser og regler
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen

3.2 Fremme og implementere algoritmisk tenkning

En naturlig måte å implementere algoritmisk tenkning er å bruke stegene i algoritmisk tenkning i problemløsning. I neste kapittel vil det redegjøres for en rekke eksempler som tar for seg dette. Til å bli mer kjent med hvordan vi kan bruke algoritmisk tenkning, vil vi her ta for oss et par enkle eksempler. Nedbrytning utgjør et kraftig verktøy som styrer hvordan vi regelmessig nærmer oss prosjekter og oppgaver, noe som også brukes i elevers læring. Eksempelene nedenfor har som mål å motivere lærere til å fremme algoritmisk tenkning i deres undervisning.

Å integrere algoritmisk tenkning i undervisningsopplegget

På høyere klassetrinn kan algoritmisk tenkning integreres i undervisningsopplegget på flere ulike måter. Her gjelder det å velge oppgaver som skaper rom for at elevene gjennomgår de fire stegene i prosessen med algoritmisk tenkning. Så lenge dette er i fokus, spiller det ingen rolle om læreren selv utformer oppgavene eller henter inspirasjon fra kollegaer, nettressurser, fagboken, og så videre. Det viktigste prinsippet er å benytte enkle oppgaver som viser essensen til begrepene som er tilknyttet algoritmisk tenkning, modellering og programmering. Nedenfor er det presentert et eksempel på en oppgave som kan tas i bruk på ungdomstrinnet, og hvordan algoritmisk tenkning inngår i disse.

Eksempel 3.2.1: *En venn har kommet med en fantastisk, men likevel kompleks matoppskrift, og ønsker nå å dele denne med verden. For å muliggjøre dette må det skrives et trinnvis og detaljert sett med instruksjoner av fremgangsmåten og tilhørende ingredienser.* I dette eksempelet skal elevene selv velge hvordan de vil gå fram for å løse oppgaven, noe som er ideelt for å fremme prosessen ved algoritmisk tenkning. I første omgang må elevene reflektere over hvilke detaljer som bør inkluderes, eller motsatt, fjerne alle unødvendige detaljer. Hvor ingrediensene skal handles inn, eller hva som er kostnadsbildet, blir mindre viktig i dette tilfellet og utgjør et viktig poeng for å løse oppgaven. Dette begrunnes med hvordan oppskriften skal være universell, og ikke begrenset av verken tilgjengelighet av ingredienser eller til et bestemt område. Arbeid med å vurdere og eliminere unødvendige detaljer, knyttes til begrepet abstrahering (se figur 3.1 og 3.2). Videre skal instruksjonene angis på en trinnvis og detaljert måte. Her må elevene være observant i hvordan oppskriften skal planlegges og settes opp slik at trinnene fremstår i korrekt rekkefølge. ◇

Eksempel: Angi fremgangsmåten på å lage HMS-prosedyrer hvis det blir brann på skolen. La elevene reflektere over: Hva var gode og mindre gode instruksjoner? Hadde rekkefølgen betydning?

Undervisningsaktivitetene som kan utfordre elever til å bruke algoritmisk tenkning:

- Enkle oppgaver/problemstillinger som kan invitere elevene til å sette opp nøkkelementene i algoritmisk tenkning.

- Aktiviteter der elevene kan drøfte sammen hvordan de kan bruke nøkkelementene i algoritmisk tenkning i en bestemt aktivitet.

3.3 Eksempler på algoritmisk tekning i forskjellige fag

Eksempel på oppgave i matematikk:

Elevene finner arealet med forskjellige former ved å spalte dem i trekanter/firkanter.
Elevene regner ut de ukjente fra de kjente og opplysningene man får i en tekstoppgave.
Elevene finner mønster i et regnestykke som kan hjelpe å forenkle beregningen, for eksempel:

$$1 + 2 + 3 + \cdots + 198 + 199 + 200$$

Språk: Elevene kan få en oppgave med tekstanalyse.

Engelsk: Elevene analyserer temaer i en tekst ved først å svare: Hvem er hovedpersonen og antagonistene? Hvor er innstillingen? Hva er konflikten? Hva er oppløsningen?

Vitenskap: Elevene forsker på de forskjellige organene for å forstå hvordan menneskekroppen fordøyer mat eller hvordan energioverskuddet lagres i kroppen.

Samfunnsfag: Elevene utforsker en annen kultur ved å studere tradisjonene, historien og normene som utgjør den.

Kunst: Elevene jobber med å bygge scenen til et teaterstykke ved å gjennomgå scenene for å bestemme deres setting og rekvisittbehov.

Eksempel 3.3.1: Bestem summen alle naturlige tall fra og med 1 til og med 200.

Løsning:

Problem: Vi skal bestemme summen:

$$S = 1 + 2 + 3 + \cdots + 198 + 199 + 200$$

1) Dekomponering/Nedbrytning: Deler problemet i mange delproblemer:

$$1 + 2 = 3$$

$$3 + 4 = 7$$

$$\vdots$$

$$199 + 200 = 399$$

Dette fører ikke til noe som gjør problemstillingen lettere.

Men hvis vi deler opp problemet slik at vi summerer det første og det siste tallet, det neste tallet og det nest siste tallet, og videre:

$$1 + 200 = 201$$

$$3 + 198 = 201$$

$$\vdots$$

$$100 + 101 = 201$$

2) Mønstre - gjenkjenne og utnytte likheter: Vi ser at vi har 100 ganger av samme sumverdi og dermed blir summen:

$$S = 201 \cdot 100 = 20100$$

3) Abstraksjon (fjerne unødvendige detaljer): Vi kan bare tenke at

$$1 + 200$$

og dette gjentar seg 100 (halvparten av antall ledd fra 1 til 200) ganger og dermed:

$$S = (1 + 200) \cdot \frac{200}{2}$$

4) Algoritme/Generalisering:

$$\text{Mer generalisering: } 1 + 2 + 3 + \cdots + (n-1) + n = (1+n) \cdot \frac{n}{2}$$

$$1 + 2 + 3 + \cdots + a_{n-1} + a_n = (a_1 + a_n) \cdot \frac{n}{2}$$

der $d = a_n - a_{n-1}$ differansen mellom to påfølgende ledd er konstant (aritmetisk rekke).

Kommentar: Her kunne tenkes også slikt:

$$S = 1 + 2 + 3 + \cdots + 198 + 199 + 200$$

$$S = 200 + 199 + 198 + \cdots + 3 + 2 + 1$$

Legg sammen disse, og vi får da:

$$2S = 201 + 201 + 201 + \cdots + 201 + 201 + 201$$

Dermed;

$$2S = 201 \cdot 200$$

$$S = 201 \cdot \frac{200}{2} = 20100$$

Oppg. 3.3.1: Her kan man jobbe i grupper. Implementer begrepene til algoritmisk tenkning forfølgende:

- a) Du skal lage påskeboller. Lag en oppskrift der de stegene i algoritmisk tenkning tas med.
- b) Du skal male huset ditt utvendig. Lag et arbeidsskjema der de stegene i algoritmisk tenkning tas med.

Oppg. 3.3.2: Oppgave:

Sett opp en liste med delemner/tema fra ditt fag som kan forbindes med algoritmisk tenkning.

Velg en av disse og lag to problemstillinger du kan tenke deg å gi til elevene for å fremme algoritmisk tenkning.

Skisser/lag en løsningsmetode for disse to problemstillingene.

Oppg. 3.3.3: Regn ut disse

- a) $1 + 2 + 3 + \dots + 9 + 10$
- b) I forrige oppgavedel kan man tenke at summen av det første og siste leddet, og tilsvarende ledd, som den andre og det neste siste leddet, er lik hverandre og da kan man gange denne summen med antall ganger denne summen dukker opp i summen. Bruk denne metoden ved hjelp av algoritmisk tenkning til å bestemme denne summen:

$$2 + 4 + 6 + \dots + 28 + 30$$

Kapittel 4

Programmering og problemløsning

Implementering av programmering og koding i skolen kommer som en følge av hvordan samfunnet vårt stadig er under digital og teknologisk utvikling. Det at samfunnet digitaliseres, innebærer blant annet at man utvikler nye tjenester, for eksempel automatisk behandling av ulike typer søknader (for eksempel barnehageplass, byggetillatelse), eValg eller elektronisk selvangivelse. I utdanningssektoren brukes apper, produkter og tjenester for å støtte undervisningen på ulike måter. Store datasett (Big Data) gjør det mulig å skreddersy tjenester og markedsføring. De siste årene har det vokst fram en internasjonal bevegelse for å fremme programmering i skolen.

4.1 Hvorfor programmering?

Dypere forståelse av underliggende prosesser og systemer, evne til logisk tenkning og ferdigheter som å være skapende og produserende, har alltid vært ansett som svært verdifull og viktige i læring og utdanning. Utviklingen av dagens teknologirike samfunn, så vel som fremtidens samfunn, utfordrer både måten vi lærer på og hvilke kompetanser som anses til å være nødvendige. Å skape meningsfulle læringsprosesser, utvikle skapende evner og dyp forståelse krever andre tilnærminger i teknologirike omgivelser. Det har blitt argumentert for hvordan forståelse og kompetanse i programmering utgjør nøkkelelementet for å kunne skape og produsere noe digitalt.

En viktig forutsetning for å lykkes med programmering i skolen er å legge opp til en struktur som sørger for at elevene får størst mulig utbytte av undervisningen. Her påpekes det hvordan dette ikke nødvendigvis er det samme som å programmere så mye som mulig og i flest mulig språk.

4.2 Hva er et program?

Grovt forklart er et program egentlig ikke noe annet enn en sekvens av kommandoer som en ønsker at programmeringsspråket (i dette tilfellet, programmet Python) skal utføre. Når det er sagt, så er det samtidig to viktige poeng som gjør at virkelige programmer fremkommer hakket mer avanserte i forhold til kommandosekvenser. I første omgang handler det om hvor viktig det er å lære seg *løkker* og *betingelser*. Løkker er et sett eller sekvenser av kommandoer som vi ønsker at maskinen skal gjennomføre et gitt antall ganger. Som oftest skilles det mellom to kategorier av løkker: *while*-løkker og *for*-løkker. I noen tilfeller benyttes det også *if*-setninger for å oppnå samme hensikt. Det andre poenget som skiller virkelige programmer fra enkle kommandosekvenser, er at utregningen som skal utføres på et bestemt tilfelle i programmet, kan avhenge av resultatet av tidligere beregninger gjennomført i programmet.

Forutsetning for å forstå algoritmer

Å forstå hvordan algoritmer fungerer kan bidra til å skape en større og bredere forståelse av digitalisering og hvordan teknologi fungerer. Innenfor matematikk og informatikk er en *algoritme* en presis beskrivelse av en endelig sekvens av operasjoner som skal utføres for å løse et problem eller et sett bestående av flere problemer. Et dataprogram kan sammenlignes med et hus bestående av algoritmer, som utgjør husets byggesteiner ideene, og konkretiseres gjennom programkoden. Algoritmer styrer eksempelvis hva som skjer når vi søker etter informasjon på internett, så vel som når vi handler på nett eller foretar transaksjoner i nettbanken. All teknologi avhenger og styres av algoritmer. Samtidig vil vi også innse hvordan algoritmer er noe som vi allerede gjør automatisk og analogt i en rekke hverdagssituasjoner. I enkelte modelleringsoppgaver er formålet å finne/observere struktur/mønster i utvikling av en figur. Med andre ord innebærer dette at man skal finne en regel. Hensikten er å fremme og tilrettelegge algoritmisk tenkning som kan være utgangspunkt for modellering. Her tar vi for oss et par eksempler.

4.3 Eksempler

programmering for matematikk

programmering for modellering

Her vil vi først ta for oss to eksempler med *programmering for matematikk*, der det ikke er særlig behov for å implementere algoritmisk tenkning og utføre avansert programmering. Videre redegjøres det for noen eksempler (*programmering for modellering*) på hvordan man kan implementere algoritmisk tenkning i lys av å kunne beherske og utvikle ferdigheter innenfor modellering.

Programmeringseksempel (programmering for matematikk)

Eksempel 4.3.1: Sett opp et python program som bestemmer middelvei og standardavvik til måleverdiene: 2.50, 3.50, 2.00, 2.85, 2.50, 5.15, 6.15, 2.25

```
1 import numpy
2 import statistics
3
4 x = [2.50, 3.50, 2.00, 2.85, 2.50, 5.15, 6.15, 2.25]
5
6 Md1x = sum(x) / len(x)
7 print("Summen og middelveien er " + str(round(sum(x), 2)) + " og " + str(
8     round(Md1x, 2)))
9 # eller
10 Md2x = statistics.mean(x)
11 Sd2x = statistics.pstdev(x)
12 print("Middelveien er " + str(round(Md2x, 2)))
13 print("Standardavviket er " + str(round(Sd2x, 2)))
14 # eller
15 Md3x= numpy.mean(x)
16 Sd3x= numpy.std(x)
17 print("Middelveien er " + str(round(Md3x, 2)))
18 print("Standardavviket er " + str(round(Sd3x, 2)))
```

```
1 Summen og middelveien er 26.9 og 3.36
2 Middelveien er 3.36
3 Standardavviket er 1.41
4 Middelveien er 3.36
5 Standardavviket er 1.41
```

Eksempel 4.3.2: Sett opp et python program som kan løse andregradsligningen $ax^2+bx+c=0$ for $a=1, b=1$, og $c=-6$.

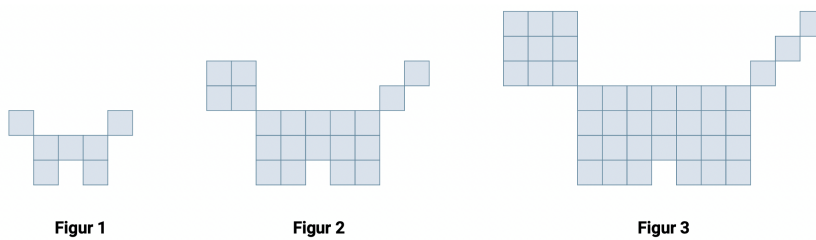
```
1 import math
2
3 # Funksjonen finner til andregradsligning
4 def andregradslign( a, b, c):
5
6     # Beregner delta (diskriminanten)
7     delta = b**2 - 4* a * c
8     sqrt_val = math.sqrt(abs(delta))
9
10    # Sjekker delta:
11    if delta > 0:
12        print(" Reelle distinkte løsninger")
13        print((-b + sqrt_val)/(2 * a))
14        print((-b - sqrt_val)/(2 * a))
15
16    elif delta == 0:
17        print("Dobbel løsning")
18        print(-b / (2 * a))
19
20    # when discriminant is less than 0
21    else:
22        print("Komplekse røtter")
23        print(- b / (2 * a), " + i", sqrt_val)
24        print(- b / (2 * a), " - i", sqrt_val)
25
26    # a, b og c settes opp
27    a = 1
28    b = 1
29    c = -6
30
31    # If a is 0, da er første gradsligning
32    if a == 0:
33        print("a=0, her har vi førstegradsligning. Rett opp!")
34
35    else:
36        andregradslign(a, b, c)
```

Ved å kjøre programmet får man:

```
1 Reelle distinkte løsninger
2 2.0
3 -3.0
```

Programmering for modellering (eksempel eksamen 1T)

Se figurene under:



Første steg i problemløsning: **forstå problemet.**

Hva er spørsmålet?

Svar: spørsmålet kan være:

Bestem arealet til bildet av hver hund.

Hva er antall kvadrater i den neste figuren? Eller bestem en formel som beregner antall kvadrater i de neste figurene.

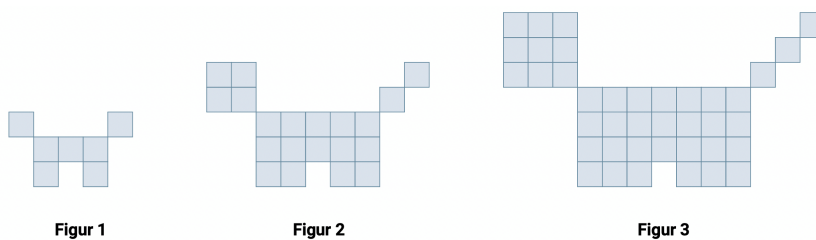
Oppgave

Ovenfor ser du tre figurer. Figurene er satt sammen av små kvadrater.

Tenk deg at du skal fortsette å lage figurer etter samme mønster.

a) Lag en algoritme som du kan bruke til å bestemme hvor mange små kvadrater du totalt trenger for å lage de 100 første figurene.

b) Bruk et programmeringsspråk, og lag et program med utgangspunkt i algoritmen. Programmet skal beregne og skrive ut hvor mange kvadrater du trenger.

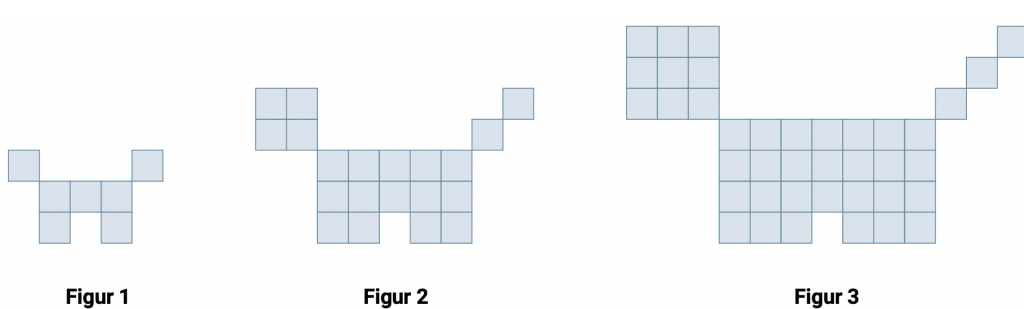


Løsning

Implementering av algoritmisk tenkning

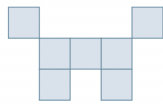
- *Logikk* - analysere
Her ser vi at figuren har forskjellige struktur når figuren utvikler/utvider seg.
- *Algoritmer* - lage steg-for-steg-beskrivelser og regler - Her ser vi at vi kan finne en regel for hver del, og kan deretter sette dem sammen.
- *Dekomponering* - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
 - Dekompontering
 - Hode: ...
 - Hale: ...
 - Foran: ...
 - Midten: ...
 - Bak: ...
- *Mønstergjenkjenning* - gjenkjenne og utnytte likheter
- *Abstrahere/Abstraksjon* - fjerne unødvendige detaljer
- *Evaluering* - vurdere resultat og prosessen

Modellering - å finne struktur (algoritmisk tenkning)

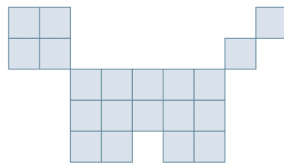


Hode: 1, 4, 9, ...
Hale: 1, 2, 3, ...
Foran: 2, 6, 12, ...
Midten: 1, 2, 3, ...
Bak: samme som foran

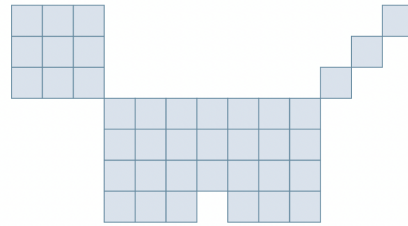
Modelling - Generalisering



Figur 1



Figur 2



Figur 3

Hode: $1, 4, 9, \dots, n^2$

Hale: $1, 2, 3, \dots, n$

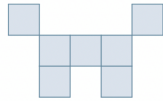
Foran: $2, 6, 12, \dots, n(n+1)$

Midten: $1, 2, 3, \dots, n$

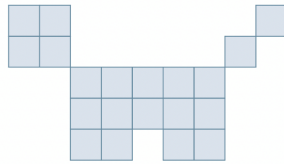
Bak: $2, 6, 12, \dots, n(n+1)$

$$n^2 + n + n + n(n+1) + n(n+1) = 3n^2 + 4n$$

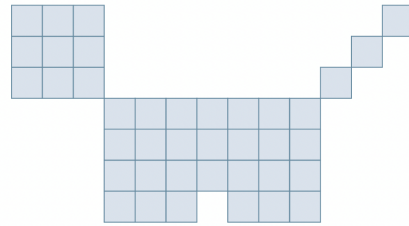
Modellering - Programmering



Figur 1



Figur 2



Figur 3

Areal:

$$A(n) = 3n^2 + 4n$$

$$n = 1 \text{ gir } A(1) = 3 \cdot 1^2 + 4 \cdot 1 = 7$$

$$n = 2 \text{ gir } A(1) = 3 \cdot 2^2 + 4 \cdot 2 = 20$$

$$n = 3 \text{ gir } A(1) = 3 \cdot 3^2 + 4 \cdot 3 = 39$$

```
1 import numpy as np
2 N = 7
3 a = np.zeros(N+1, np.int64)
4 a[0] = 1
5 for n in range(1, N+1):
6     a[n] = 3*n**2+4*n
7     print(n, a[n])
```

Modellering - testing

Ved å kjøre programmet får vi:

```
1 1 7
2 2 20
3 3 39
4 4 64
5 5 95
6 6 132
7 7 175
```

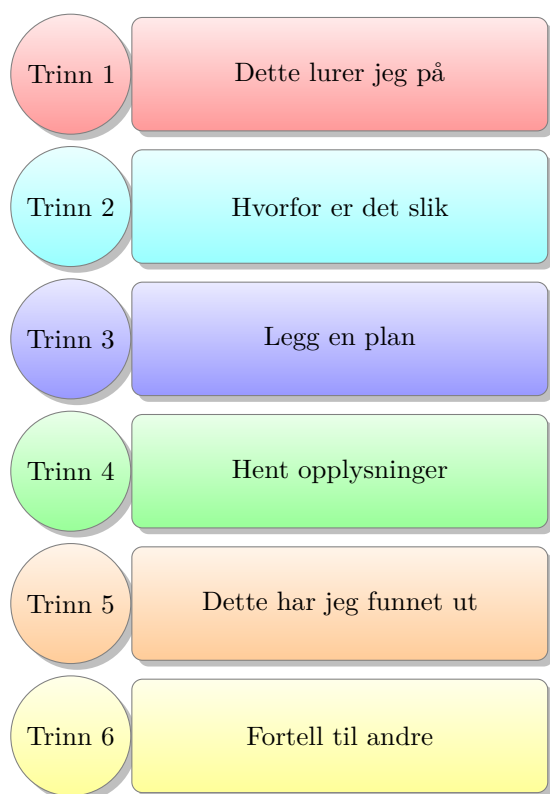
Her kunne man vurdere om man kunne forbedre/endre/slette noe i prosessen.

◇

Nysgjerrigpermetoden (barnetrinnet)

Nysgjerrigpermetoden [5] er en utforskende læringsmetode for grunnskolen, som ble utviklet av Forskningsrådet i 1996. Nysgjerrigpermetoden er en læringsform der elevene deltar aktivt og får ansvarsfulle roller. Nysgjerrigpermetoden er en seks-trinns «oppskrift» på hvordan man kan jobbe etter vitenskapelig metode i barneskolen. Metoden er en forenklet utgave av hypotetisk-deduktiv metode, som benyttes både i natur- og samfunnsvitenskap. Elevene blir utfordret til å stille spørsmål, lage egne hypoteser og gjøre egne undersøkelser. Når man jobber etter Nysgjerrigpermetoden, trekkes gjerne flere fag inn, og man får trent alle de grunnleggende ferdighetene. Når elevene selv får forske på noe de virkelig er nysgjerrige på, er de motivert for læring.

Nysgjerrigper-metoden er utviklet særlig for barnetrinnet, og består av følgende seks-trinn:



Med Nysgjerrigpermetoden vil en:

- gjøre lærere kjent med at vitenskapelig metode kan brukes som arbeidsmetode i alle fag
- vise elever hvordan kunnskap blir til og hvordan de selv kan finne svar på spørsmål gjennom egne undersøkelser.

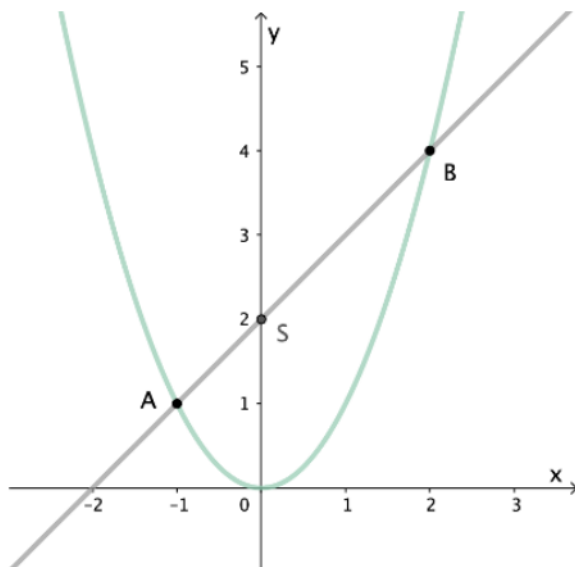
Forskningsmetoden

Martinez og Stager [?] (s. 46) forklarer at i framtiden trenger man ikke forskere som har lært seg mengder av faglig stoff utenatt. Det trengs kreative, selvstendige problemløsere på alle områder. Dermed har man noe som man kaller for forskningsmetoden. Den går ut på ulike steg, slik som:

- Å observere noe, eller gjøre et søk
- Lage en hypotese
- Lage en antagelse som er basert på hypotesen

- Teste hypotesen ved å gjøre et eksperiment
- Analysere resultatet av eksperimentet
- Fastslå om hypotesen var riktig.

Eksempel 4.3.3: Plasser to punkter A og B på parabelen gitt ved $f(x) = x^2$ og trekk en linje gjennom dem.



- Finne en sammenheng mellom x -koordinatene til punktene A og B til skjæringspunktet mellom linjen og y -aksen.
- Vis at sammenhengen gjelder for alle par av punkter A og B på parabelen.

Løsning: En matematiker vil løse den på denne måten:

a) $y = x + 2$

b) $x^2 = x + 2$ som gir $x = -1$ og $x = 2$ Generalisering: $x^2 = ax + b$ som gir $x^2 - ax - b = 0$ som gir to x -verdier for alle par: $x = \frac{a \pm \sqrt{a^2 - 4b}}{2}$ der $a^2 - 4b > 0$.

Men man trenger tid for å gå gjennom denne oppgaven for elevene. Dette er en typisk modelleringseksempel som kan brukes for elevene at de kan tenke selv og diskuterer sammen for å komme til løsningen.

Oppgave: Bruk algoritmisk tenkning/modellering/problemløsning til å sette opp løsningsmetode for forrige eksempelet. $\diamond$

Oppsummering - viktige begrep

Problemløsning: handler om å forstå problemet, dele opp problemstillingen, analysere fakta og opplysningene, prøve å finne en tilpasset løsningsmetode, løse problemet, reflektere, vurdere hvor gyldig løsningen er, og evaluere løsningsstrategien, og hele problemløsningsprosessen,.

Algoritmisk tenkning: er en problemløsningsmetode. Algoritmisk tenkning innebærer å tilnærme seg problemer på en systematisk måte, både når vi formulerer hva det er vi ønsker å løse, og når vi foreslår mulige løsninger.

Modellering: handler om aktivitet og prosessen for forenkling av virkeligheten som brukes for å beskrive tilstander og egenskaper som det er vanskelig eller umulig å definere/måle direkte. Modellering derier seg om å kunne omformulere en problemstilling fra virkeligheten, til et matematisk problem, løse dette problemet og så vurdere gyldigheten og begrensningen til løsningen i den opprinnelige situasjonen.

Matematisk modellering: handler om å kunne omformulere en problemstilling fra virkeligheten, til et matematisk problem, løse dette problemet og så vurdere gyldigheten og begrensningen til løsningen i den opprinnelige situasjonen.

Programmering: er prosessen med å lage et sett med instruksjoner som forteller en datamaskin hvordan du skal utføre en oppgave.

Koding: er bare den delen av programmeringen som handler om å fortelle en datamaskin hva den skal gjøre. Koding i skolen dreier seg om at elevene lærer å løse problemer og samarbeide på kreative måter. Koding er brukt i mange sammenhenger i skolen som et begrep som omfatter programmeringsaktiviteter.

Realfaglig programmering og matematisk modellering handler om problemløsning, algoritmer, logisk tenkning og argumentasjon. Matematikk utgjør hovedtyngden av opplæringen, men programmering kommer inn i flere fag. Kunnskap om numeriske metoder og realfaglig programmering åpner for å utforske fagene mer i dybden, og tilrettelegger for både bredde- og dybdelæring. I programmering får elever bruk for kunnskap om numeriske metoder, som er matematiske løsningsteknikker tilpasset datamaskinen.

Kapittel 5

Programmering som støtte for matematikkundervisningen

Å skape meningsfulle læringsprosesser kan være nøkkelement for å integrere programmeringen i matematikkundervisningen.

Det kan være hensiktsmessig å stille opp følgende spørsmål:

- Hvordan knyttes programmering opp til matematikk på skolen nå?
- Hvordan kan programmering støtte matematikkundervisningen?
- Hvordan benytte programmering til å lære bort matte?
- Hvordan lære bort programmering på en god måte?
- Kan programmering gjøre matematikk mindre abstrakt?

Her tas noen eksempler som kan prøve å vise hvordan algoritmisk tenkning og programmering kan integreres i matematikkfaget uten at den skal stjele tiden og tvert imot hjelper til å gjøre matematikken tilgjengelig for praktiske situasjoner. Det kreves ikke å kunne skrive programkode men å kunne beskrive hva en programkode utfører, kunne gjør de nødvendige endringene, og eventuelt tilpasse programmet til en ny problemstilling.

Oppg. 5.0.1: Hva utfører følgende program?

```
1 km = float(input("Hva er reiselengden (KM): "))
2 p= int(input("Antall passasjerer: "))
3 # Kostand regning
4 if km<5:
5     kostnad=250
6 else:
7     kostnad = 200 + ((km)*10)
8 # Hvis antall passasjerer er mer enn 4, da må bestille maxi-taxi som
   koster 1.25 ganger mer.
9     if p >= 5:
10        kostnad = kostnad * 1.25
11 # Den totale kostnaden
12 print(f'Reisekostnaden for {p} passasjerer er Kr.{kostnad:.2f}')
```

Kan du skrive et lignende program for et lignende problem der et mobilabonnement der han betaler

- 100 kroner hvis sender mindre enn 400 meldinger pr. mnd
 - ellers betaler fastpris 50 kr. og 0.1 kr. per melding, og hvis sende
 - Hvis man sender mer enn 1000 meldinger pr. måned, får man fast pris 150 kr.
- man kan ringe gratis til alle i dette abonnementet.
Hvor mye betaler man per måned hvis han sender x antall meldinger?

Oppg. 5.0.2: Liva har ni gullmynter som ser like ut. Nøyaktig en av dem er falsk og har en annen vekt enn de andre (du vet ikke om den er lettere eller tyngre). For å finne den falske gullmynten skal du bruke en enkel skålvekt. Tuva påstår at hun kan finne den falske gullmynten ved hjelp av bare tre veiinger. (Ved hver veiing legger du perler i hver av skålene. Vekten forteller deg bare hvilken skål som har den tyngste lasten.)



a) Har Tuva rett? Begrunn svaret.

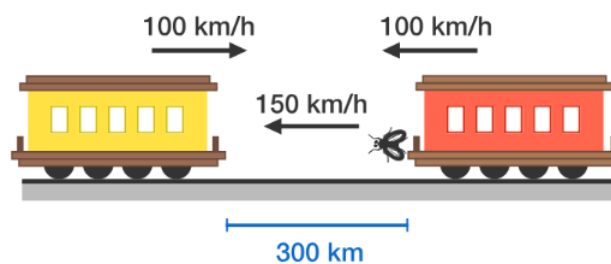
Tenk nå at Liva har 12 gullmynter og en var falsk og har en annen vekt enn de andre (du vet ikke om den er lettere eller tyngre).

b) Med minst hvor mange veiinger kan man finne den falske?

Oppg. 5.0.3: To tog som hver kjører med en hastighet på 100 km/t er på vei mot hverandre på samme spor. I det nøyaktige øyeblikket er togene 300 km fra hverandre begynner en superrask flue foran på ett tog å fly frem og tilbake mellom de to togene med en hastighet på 150 km/t. Hva er den totale avstanden fluen vil ha tilbakelagt før den ble knust mellom togene?



Oppg. 5.0.4: To tog som hver kjører med en hastighet på 100 km/t er på vei mot hverandre på samme spor. I det nøyaktige øyeblikket er togene 300 km fra hverandre begynner en superrask flue foran på ett tog å fly frem og tilbake mellom de to togene med en hastighet på 150 km/t. Hva er den totale avstanden fluen vil ha tilbakelagt før den ble knust mellom togene?



Oppg. 5.0.5:

a) Hva utfører følgende program?

```
1 import numpy as np
2
3 r= float(input('Skriv inn radiusen til en sirkel: '))
4
5 def A(r):
6     return np.pi*r**2
7
8 print(A(r))
```

Hva blir svaret hvis man skriver som svar 1 når man kjører programmet?

b) Skriv et program som leser fra brukeren radius og høyden til en rett sylinder og regner ut volumet, og printer ut resultatet.

Fasit

Fasit 5.0.1:

En drosjeselskap har følgende regler for å regne reisekostnader:

- Hvis reiselengden er mindre enn 5 km., er fast pris 250 kr.
- Fast start pris er 200 kr. For hver km. regnes 10 kr.
- Hvis antall passasjerer er mer enn 4, må bestilles maxi taxi som koster 1.25 ganger mer.

Skriv en programkode som leser inn reiselengden og antall passasjerer fra brukeren, og regner ut reisekostnaden og skriver ut resultatet.

```
1 x = float(input("Hvor mange sms er sendt? "))
2
3 # Kostand regning
4 if x<500:
5     kostnad=100
6 else:
7     kostnad = 50 + (x)*.1
8 # Hvis antall sms >=1000, er kostnaden 150 kr.
9     if x >= 1000:
10        kostnad = 150
11 # Den totale kostnaden
12 print(f'Mobilkostnaden for {x} sms er Kr.{kostnad:.2f}')
```

Fasit 5.0.2:

Tenk selv.

Fasit 5.0.3:

Tiden til togene treffer hverandre: $t = \frac{d}{v_1 + v_2} = \frac{300}{100 + 100} = 1,5$ time.

I løpet av denne tiden har fluen flyd $x = v_3 \cdot t = 150 \cdot 1,5 = 225$ km.

Det tar for fluen som har relativ hastighet $v = v_2 + v_3 = 250$ km/t å komme til toget som

kommer mot fluen, $t_3 = \frac{d}{v_2 + v_3} = \frac{300}{250} = 1,2$ time, og da flyr tilbake 0,3 t før den blir truffet.

Fasit 5.0.4:

Tiden til togene treffer hverandre: $t = \frac{d}{v_1 + v_2} = \frac{300}{100 + 100} = 1,5$ time.

I løpet av denne tiden har fluen flyd $x = v_3 \cdot t = 150 \cdot 1,5 = 225$ km.

Det tar for fluen som har relativ hastighet $v = v_2 + v_3 = 250$ km/t å komme til toget som

kommer mot fluen, $t_3 = \frac{d}{v_2 + v_3} = \frac{300}{250} = 1,2$ time, og da flyr tilbake 0,3 t før den blir truffet.

Fasit 5.0.5:

- a) Programmet leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel. Svaret blir 3.14

b)

```
1 import numpy as np
2
3 r= float(input('Skriv inn radiusen til en sylinder: '))
4
5 h= float(input('Skriv inn høyden til en sylinder: '))
6
7 def V(r,h):
8     return np.pi*r**2*h
9
10 print(f' Volumet til en sylinder med radius {r} og {h} er lik {V(r,h)}')
```

Bibliografi

- [1] BBC Bitesize (2019). KS3 Computer Science. Introduction to computational thinking. Hentet fra <https://www.bbc.co.uk/bitesize/topics/z7tp34j>
- [2] Udir (2019). Matematikk fellesfag 1-10 – oppsummering av høringen. Hentet fra <https://www.udir.no/laring-og-trivsel/lareplanverket/fagfornyelsen/oppsummeringer-av-7000-horingssvar-om-nye-lareplaner/matematikk-grunnskole-oppsummering-av-horingen/>
- [3] Martinez, S. L & Stager, G. S. (2019). Invent to learn: Making, Tinkering and Engineering in the Classroom (2. Utgave). Torrance, California, USA: Constructing Modern Knowledge Press
- [4] Jakobsen, Renate E. (2019) Programmering i matematikk –muligheter og utfordringer (Masteroppgave, UiA). Henter fra <https://uia.brage.unit.no/uia-xmlui/bitstream/handle/11250/2646381/Renate20Elise20Jakobsen.pdf>
- [5] Stenstad, T. & Løken, M. (2006). Nysgjerrigpermetoden: Vitenskapelig arbeidsmetode i barneskolen: Veiledning for lærere. Hentet fra <https://www.forskningsradet.no/siteassets/publikasjoner/1254011388699.pdf>
<https://www.forskningsradet.no/siteassets/publikasjoner/1254022501276.pdf>
- [6] Biologiske fagtermer fra A til Å. Institutt for biovitenskap ved matematisk-naturvitenskapelig fakultet ved UiO. Hentet fra <https://www.mn.uio.no/ibv/tjenester/kunnskap/plantefys/leksikon/>.
- [7] Collins Online Dictionary. Hentet fra <https://www.collinsdictionary.com/dictionary/english>
- [8] Kunnskapsdepartementet (2013). Læreplan i matematikk fellesfag. Hentet fra <https://www.udir.no/kl06/mat1-04>
- [9] Kunnskapsdepartementet (2018). Kjerneelementer i fag. Hentet fra <https://www.regjeringen.no/no/aktuelt/forny-er-innholdet-i-skolen/id2606028/>
- [10] Papert, Seymour. Mindstorms: Children, computers, and powerful ideas. Basic Books, Inc., 1980. Hentet fra <http://worrydream.com/refs/Papert20-20Mindstorms201st20ed.pdf>
- [11] Programmering som kompetanse for det 21. århundre. Udir, Notat nr. 2 - Programmering i skolen. November 2016. <https://www.udir.no/kvalitet-og-kompetanse/profesjonsfaglig-digital-kompetanse/notat-om-programmering-i-skolen/>
- [12] Utdanningsdirektoratet (2013). Læreplan i matematikk fellesfag. Hentet fra <https://www.udir.no/kl06/MAT1-04>
- [13] Utdanningsdirektoratet (2017). Kjerneelementer i matematikk, men hvorfor programmering? Hentet fra <https://udirbloggen.no/kjerneelementer-i-matematikk-men-hvorfor-programmering/>
- [14] Utdanningsdirektoratet (2019). Læreplan i matematikk 1.-10. trinn. Hentet fra <https://www.udir.no/lk20/mat01-05>
- [15] Utdanningsdirektoratet (2019). Læreplan i matematikk fellesfag Vg1 teoretisk. Hentet fra <https://www.udir.no/lk20/mat09-01?lang=nob>
- [16] Wing, Jeannette (april 2008). «Computational thinking and thinking about computing». 2008 IEEE International Symposium on Parallel and Distributed.

Dette heftet vil fremme læreglede for å lære seg algoritmisk tenkning, modellering og problemløsning, og vil bruke programmering til å vise hvordan man kan løse oppgaver og problemer fra realfag og andre fag i ungdomstrinnet og videregående skole trinnet.

Problem: Skriv en programkode som regner ut volumet til en kule med radius r , og skisserer grafen til $V(r)$, for $0 \leq r \leq 5$.

Matematisk modell: $V(r) = \frac{4}{3}\pi r^3$

Programmering:

```
1 import matplotlib.pyplot as plt
2 from math import pi
3
4
5 # Leser opp radiusen fra skjermen
6
7 radius = float(input('Radius til kule:'))
8
9 # Funksjon som regner ut volumet til en kule med radius r
10
11 def volum(radius):
12
13     return (4/3) * pi * radius **3
14
15 V=volum(radius)
16
17 # Leser opp radiusen fra skjermen
18
19 print(f"Volumet til en kule med radius {radius} er: ", V)
20 r_verdier = np.arange(0,5.2,.2)
21 print(r_verdier)
22
23
24 #for r in r_verdier:
25     # formatering     print(f"{x:.2f}\t{f(x):.2f}")
26
27 r=r_verdier
28 V=volum(r)
29
30
31 #print(f'{r:.2f}\t,{V:.2f}')
32
33 plt.plot(r,V,'.-', linewidth=3, label='V(r)')
34 plt.grid()
35 plt.xlabel('r')
36 plt.ylabel('V')
37 plt.legend(loc='best')
38 plt.show()
```

Programmeringsdidaktikk: Struktur og progresjon i oppgavene skal forberede deltagere til å kunne løse utfordrende problemstillinger og tilegne seg utholdenhet, kreativitet, selvtillit, og vil bidra til matematisk tenkning.

Problem: vi skal se på filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperaturmåling i en te-kopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 Grad celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

Matematisk modell:

$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i Grad celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

Programmering:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hib.no/ansatte/ahas/MP/tekopp-temp.
   csv', sep=';', decimal='.', comment='#')
5
6 tid = df['Tid'].tolist()
7 temperatur = df['Temperatur'].tolist()
8 tstart = 0
9 Tstart = temperatur[0]
10 tslutt = 3700
11 deltat = 0.5
12 To = 22 # Rom temperatur
13 k = .0015 # proporsjonalitets koeff.
14 tverdier = []
15 Tverdier = []
16 t = tstart
17 T = Tstart
18 while t <= tslutt:
19     tverdier.append(t)
20     Tverdier.append(T)
21     t = t + deltat
22     deltaT = -deltat * k * (T - To)
23     T = T + deltaT
24
25 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
26 plt.plot(tid, temperatur, 'x', color='red')
27 plt.grid()
28 plt.xlabel('t (sek)')
29 plt.ylabel('T (Grad celsius)')
30 plt.legend(loc='best')
31 plt.show()
```

